ECE/CS 285 Advanced Image Synthesis

HW 1 – REYES-Style Rendering System

Due Wednesday, October 21, 11:59pm

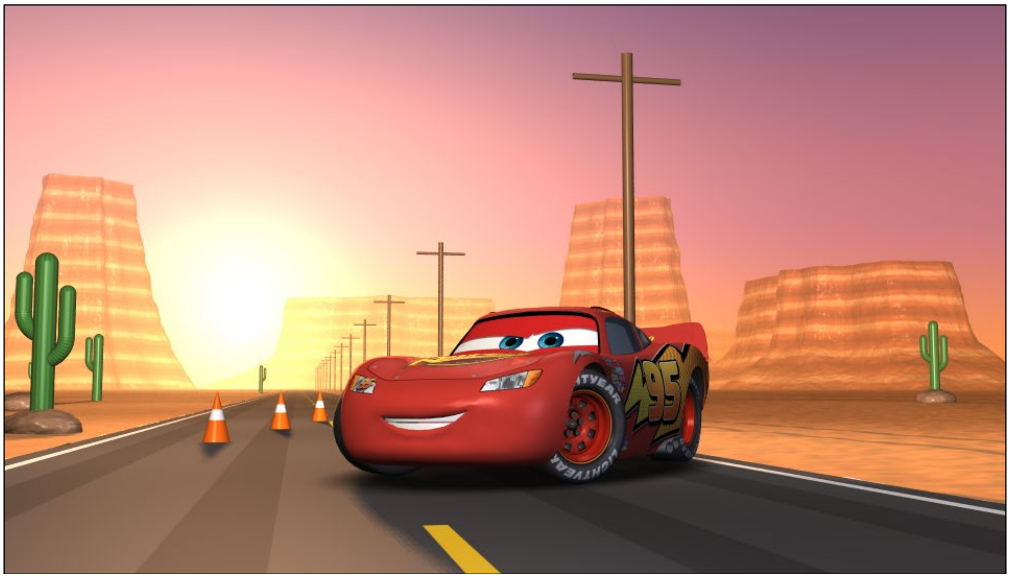


Figure 1. The `RadiatorSprings` scene that you will be able to render with your finished REYES rendering system. Lightning McQueen character © Pixar. AI tools were used to help generate this scene.

1. Overview

In this assignment, you will implement a REYES-style, micropolygon-based renderer in **C, C++, or Rust**, driven through a RenderMan-like C interface. This assignment assumes familiarity with vectors and matrices, transformations, basic data structures, and your chosen implementation language. It implements only a subset of RenderMan, but enough to render nice pictures like the one in Fig. 1, and enough to show you how a REYES renderer such as PRMan works: set up a graphics state, dice each primitive into a grid of micropolygons, shade the grid, bust it into micropolygons, and resolve visibility with the stochastically sampled z-buffer (the "hider") from the REYES paper by Cook et al.

Scenes are all C++ functions that call `Ri*()` functions directly (although most modeling programs write RIB files, we use the C bindings instead for ease of implementation). Everything the scenes call has C linkage, so both language kits use the same scene sources. For C, replace the renderer and shader source entries in `CMakeLists.txt` with your `.c` implementations and adapt the supplied C++ image/depth writers as described in `README.md`; the scene files remain C++. A Rust renderer exports the same C interface, and the kit's `Cargo` project compiles and links the C++ scene files against it.

For example, here's a scene that draws a simple sphere:

```
void FirstSphere(void){
     float fov = 40.0f; /* fov will be 40 degrees */
     RiBegin(RI_NULL);
         RiFormat(320, 240); /* set the image resolution */
         RiDisplay("FirstSphere.png", "", ""); /* set output filename/type */
         RiPixelSamples(1, 1); /* 1 x 1 samples/pixel */
         RiFrameBegin(0);
```

```
            RiProjection(RI_PERSPECTIVE, "fov", &fov);
            RiWorldBegin();
                RiSurface(CONSTANT, NULL, 0); /* flat, no shading at all */
                RiTranslate(0, 0, 5); /* 5 units in front of the eye */
                RtColor red = {0.9f, 0.2f, 0.2f};
                RiColor(red);
                RiSphere(1, -1, 1, 360);

            RiWorldEnd();
        RiFrameEnd();
    RiEnd();
}
```

You will implement many (but not all) `Ri*()` functions to get a working system. **Write your own code** for the required renderer algorithms**:** apart from the supplied helpers and the fixed Perlin permutation table permitted in Sec. 5, **do not** copy implementation **code** from **the internet or elsewhere, and do not have an LLM write it for you** (see Sec. 16 for what AI tools may and may not do).

Starter Kit:
To get you started, the starter kit contains:

- `include/Ri.h`: The function prototypes and the shader context struct (Sec. 5). This is the interface the scene files compile against.
- `include/shaders.h`: Four shaders you must implement, their parameter structs, and `perlinNoise()`.
- `include/scenes.h`, `scenes/0_scene_table.cpp` and the scene files in `scenes/`, in three tiers (Section 8).
- `src/ri.cpp` and `src/shaders.cpp`: Stubs for your renderer and your shaders. They compile as they are, and every function has a comment saying what it must do. `writeImage()` and `writeDepth()` in `src/ri.cpp` are complete. `src/main.cpp` picks the scene by name.
- `include/obj.h`: Small OBJ loader that some scene files use to load models. It converts OBJ texture coordinates, with t = 0 at the bottom of the image, to ours, with t = 0 at the top.
- `include/shadowmap.h`: Shadow-map lookups for the scene files, built on the depth files your renderer writes (Sec. 4).
- `include/sceneutil.h`, `include/models.h` and `include/anim.h`: Helpers the scene files share (a camera built from an eye point and a target, the code that draws and poses the models). Your renderer never calls them.
- `models/`: The Lightning McQueen, Buzz Lightyear, Wall-E and Woody models and their textures, plus Gumbo, the elephant from "Advanced RenderMan," as a triangle mesh with vertex normals.
- `textures/`: The textures the scenes use, including the UV test texture.
- `third_party/`: `stb_image.h` and `stb_image_write.h`, for reading and writing images.
- `CMakeLists.txt` for C and C++, and `rust/` for Rust: `Cargo.toml`, `Cargo.lock`, `build.rs` (compiles the scene files and links them against your renderer), `src/main.rs` (the scene picker), and `src/ri.rs` and `src/shaders.rs` (renderer and shader stubs with the same public interface as `include/Ri.h` and `include/shaders.h`).
- `README.md`: Required toolchains, platform-specific build/run commands, scene selectors and the two movie-encoding commands.
- `renders/`: Where final images go, in the same three tiers as `scenes/`.
- `reference_renders/`: My renderer's image of every scene (and both movies), in the same three tiers. Compare yours against these.

Preserve the existing function signatures, exported names, and public struct layouts: the scene files compile against that interface. Implement and reorganize the renderer and shader stubs as needed. For extra credit you may add new declarations, but do not remove or change existing ones. You are free to use scenes, textures, and shader ideas from other resources to test your implementation and to model your final scene, as long as you document the sources.

For whatever implementation language you choose, follow the platform-specific build commands in the kit's `README.md`. Until you fill in the stubs, color outputs are black (fully transparent in PNG/TGA), and depth outputs contain only +infinity. Running `reyes` with no selector lists the scenes (see Sec. 8).

2. Setting up graphics state

You need to implement the commands that set up basic scene and image options, such as the image format and the parameters of the camera (position, `fov`, projection transformation). For the projection transformation (set with `RiProjection()`), you should handle both **orthographic** and **perspective** (`RI_ORTHOGRAPHIC` and `RI_PERSPECTIVE`, respectively). Note that `RiProjection()` only records the selected projection and its field of view: the hider applies it after dicing and shading, when it takes each micropolygon's camera-space vertices to screen space (Sec. 4).

The scene selects an output name with `RiDisplay()`. The supplied image writer chooses the format from the filename extension: `.png`, `.bmp`, `.tga` or `.jpg`. PNG and TGA preserve alpha; JPEG and BMP are written over black without alpha. `WallEMovie` writes JPEG frames. `RiDisplay(name, "", "")` requests a picture; so does `RiDisplay(name, "file", "rgb")`, used by `SimpleAnimation`. A type of `"zfile"` or a mode of `"z"` instead requests a depth image (Sec. 4). `RiWorldEnd()` must finish and write the requested output before returning. `RiFrameEnd()` restores the graphics state saved by `RiFrameBegin()`; it does not render or write the image.

Session and frame state:

`RiBegin()` resets color and opacity to `(1,1,1)`, the surface shader to `STANDARD` (see Sec. 5), displacement to none, the transformation to identity, projection to orthographic, image size to 640 × 480 and sampling to 2 × 2. `RiFrameBegin()` saves graphics state and `RiFrameEnd()` restores it. `RiWorldBegin()` preserves the world-to-camera transform built so far and starts a fresh sample buffer. Within the world, object transforms are composed with that camera transform. `RiWorldEnd()` finishes and writes the current image before returning. Texture slots last until `RiEnd()` or replacement of the slot.

RenderMan conventions:

RenderMan uses a **left-handed coordinate system**. The camera sits at the origin looking down **`+z`** with **`+y`** up, so objects with positive **`z`** are in front of the camera (different from the way OpenGL does it as we derived in class). The transformations issued between `RiFrameBegin()` and `RiWorldBegin()` build the world-to-camera transform, and once inside `RiWorldBegin()`, transformations apply to objects the same way. All transformations are applied to objects in **reverse order** of the calls: `RiTranslate(0,0,8)` followed by `RiRotate(-20,1,0,0)` rotates the world first, then pushes it 8 units in front of the camera. Any call to `RiProjection()` (whichever projection it selects) also resets the current transformation to the identity, so the camera transform built after it starts fresh in every frame.
Furthermore, RenderMan represents points as row vectors, so a point `p` is transformed as `p′= pM` (i.e., **postmultiplying** `p` by `M`), with the translation in the matrix's last row. This differs from the column-vector form `p′= Mp` used in the real-time pipeline discussed in class. However, you may use either matrix convention internally within your code: with column vectors, **transpose** the supplied `RtMatrix` and

postmultiply the current transformation matrix by this transpose; with row vectors, **premultiply** the current transformation matrix by the supplied matrix. But in either case, the transformation called last is applied to a point first. We can use `RiConcatTransform(A)` to combine the supplied matrix `A` with the current transformation matrix (CTM). So if `C` is the CTM before the call and `C'` its updated value, then in row-vector representation you concatenate as `C' = A C`; thus `p' = pC' = (pA)C`, so the newly supplied transformation acts first. Finally, keep in mind that mathematical row/column-vector conventions are separate from how the 16 numbers are stored in memory.

Mapping stuff to the screen:
The default screen window after projection is `[-a, a] × [-1, 1]` for `a = xres / yres ≥ 1` (i.e., a horizontal aspect ratio of `a:1`), and `[-1, 1] × [-1/a, 1/a]` for `a < 1` (i.e., a portrait aspect ratio). For a perspective projection, `fov` is the full field-of-view angle in degrees spanned by the shorter image dimension: a vertical FOV for a landscape image and a horizontal FOV for a portrait image. For perspective, project points in camera space with `x_screen = x/[z tan(fov/2)]` and `y_screen = y/[z tan(fov/2)]`; for orthographic, `x_screen = x` and `y_screen = y`. Compute the aspect ratio with floating-point division, and convert degrees to radians when evaluating trigonometric functions.

Finally, map the screen window to continuous raster coordinates `[0,xres] × [0,yres]`, with `y` reversed so row zero is at the top. Keep in mind that you will use **continuous** raster coordinates when testing micropolygons against samples: i.e., pixel `(i,j)` covers the unit square from `(i,j)` to `(i+1,j+1)`, with its jittered samples located inside that square. When projecting a grid vertex to raster space, retain its original camera-space `z` value (its depth) as well, which you'll need when determining visibility and when writing depth images.

Other things you will have to implement to get this to work:
`RiDisplay()`
`RiProjection()`

You don't have to worry about:
- Motion blur (no `RiShutter()`) or depth of field
- User-programmable clip planes
- Instanced objects
- RIB interfaces (assume everything is through the C binding)
- Restoring attributes: `RiTransformBegin()/RiTransformEnd()` save and restore the current transformation only. Color, opacity, and bound shaders are not restored and persist until changed.

3. Processing of primitives
In this assignment, you can assume that every primitive is "diceable," so you will **not have to split any primitives**. This will generate lots of micropolygons but that is ok for the assignment (performance is not a major issue, but I frown on extremely slow implementations). You will need to compute a **bound** for each primitive, determine an appropriate dicing rate, and evaluate it on a grid that will ultimately be broken into micropolygons.

Overall primitive-processing pipeline:
Every primitive follows the same basic sequence. First, compute its bound and use that bound to determine the dicing rate. Then evaluate the primitive on an object-space grid, including its geometric attributes, and transform the grid positions and normals into camera space. If a displacement shader is bound, run it over the **entire grid** and recompute the shading normals from the displaced positions. Orient the normals for lighting, then run the surface shader over the **entire grid**. Finally, project the grid to raster space, break it into micropolygons, and pass those micropolygons to the hider, which records their

contributions to the image samples. Once a primitive has been processed, free its grid before moving on to the next primitive.

Therefore, you do not need to retain the diced geometry for the whole scene**.** The grid is per-primitive working storage: once a primitive has been displaced, shaded, broken into micropolygons, and its contributions have been recorded by the hider, you can free that grid in memory before processing the next primitive. What must persist across primitives is the renderer state needed by subsequent primitives (e.g., the current graphics state and loaded textures) and the hider's per-sample visibility data, since later primitives may contribute hits in front of or behind earlier ones. At `RiWorldEnd()`, resolve and filter the accumulated sample data to produce the final image (or depth image), write it to disk, and then release the image-specific storage.

How finely to dice:
Project the eight corners of the primitive's bound to the screen and take the bounding rectangle of the projected points. If **d** is the length of that rectangle's diagonal in pixels, then you'd dice the primitive into an **n** × **n** grid with **n** = **d** / (½ pixel) = **2d**, so that even a parameter direction that runs along the diagonal gets micropolygons about half a pixel long. For a bound that projects to 100 × 100 pixels, **d** ≈ 141 pixels and **n** ≈ 283, that is, 283 micropolygons along **u** and another 283 along **v** (an **n** × **n** grid, not **n** + **n**). Round **n** up to an integer and clamp it to a minimum of 2 and a maximum of your choosing (my implementation uses 1024). The minimum is a resolution floor, not a guarantee that every distant object hits a sample. Furthermore, for a parameter direction that wraps through a full 360° (for example, the angular direction of a full sphere, cylinder, or torus), you may clamp that direction to a minimum of 4 to avoid a degenerate grid.

Note that **n** here is an estimate, not a guarantee. On a closed quadric, a parameter direction may wrap around the entire surface, so this estimate need not bound every projected edge by half a pixel. A coarser dice can affect silhouettes, displacement and shading. Adjacent grid cells should still meet along shared edges; holes also warrant checking the coverage and indexing code. Use the stated bound-based estimate for the required implementation. Measuring the real projected edge lengths and re-dicing is an extra-credit item. PRMan exposes the micropolygon size as `RiShadingRate()`, which you can ignore.

Finally, remember that we compute bounds **before displacement**. Since we don't cull by bounds in this project, an underestimated bound won't incorrectly cull an object whose bounds are outside the screen but gets displaced into view (although it can change the dice rate). However, if you add culling of primitives based on their bounds (for extra credit), you must pad the bound by the maximum displacement to account for that. My implementation pads the bound whenever a displacement shader is bound, by `max(0.5, 0.1 ×` the bound's diagonal`)`, which also makes displaced objects dice a little finer. This optional padding is a dicing heuristic, not a guaranteed maximum for arbitrary displacement shaders.

Objects that cross the eye plane:
Although some objects do cross the eye plane (e.g., in the `Explorer` scene, the ground, a small moon, reaches behind the camera), you still don't split them. Instead: (1) discard every micropolygon (whether from a quadric or a polygon) that has a vertex at $z \leq 0.01$ before you project it, since dividing by a z near or below zero throws it across the whole screen, and (2) if any corner of a primitive's bound has $z \leq 0.01$, its projection means nothing, so dice that primitive at your maximum resolution.

Geometric primitives:
You will need to support the following **five primitives (4 quadrics and one polygonal)**:

1. Sphere (`RiSphere`)
2. Cylinder (`RiCylinder`)

3. Cone (`RiCone`)
4. Torus (`RiTorus`)
5. Polygon mesh (`RiPointsPolygons`)

Follow the RenderMan specification for each (Sec. 5 of the spec, Fig. 5.3). Note that all angles (`thetamax`, `phimin`, `phimax`) are in degrees. For `RiSphere` only, clamp `zmin` and `zmax` to `[-r, r]`, since scenes may pass values outside that range (e.g., `RiSphere(1, -1.5, 1.5, 360)` is a full sphere). Also, note that the equations in Sec. 5.4 of RiSpec 3.2.1 have three typos. The sphere's third line sets **z**, not **x**. The cylinder's height is z = `zmin + v·(zmax - zmin)`. The torus's angle is φ = `phimin + v·(phimax - phimin)`. If you copy the spec's text as printed, you get a flattened sphere, a cylinder that always starts at z = `0`, and a torus that collapses to a single circle.

In terms of bounds, a conservative object-space box is sufficient (tight bounds for partial angles are optional): e.g., a sphere or cylinder fits in `x,y ∈ [-r,r]` with its valid `z` range; a cone fits in `x,y ∈ [-r,r]` and `z ∈ [0,height]`; a torus fits in `x,y ∈ [-(R+r),R+r]` and `z ∈ [-r,r]`. Transform all eight corners to camera space before applying the projection/eye-plane rule. These quadrics describe surface pieces; do not add end caps or caps over clipped parameter ranges unless a scene draws them separately.

As you evaluate each point in the micropolygon grid, you'll need to evaluate a few variables available to all shaders: **`P`** (the position of the point on the surface, in camera space), **`Po`** (the same point in object space, for procedural patterns that should stick to the object), the surface parameters **`(u,v)`**, and the surface shading normal **`N`**. These are passed to the shaders through the context struct described in Sec. 5.

Computing normals:
Before displacement, compute the normal that will be given to the displacement shader. For a quadric, compute its analytic normal in object space and transform it to camera space with the **inverse transpose** of the modelview matrix (multiplying only by MV gives incorrect results under anisotropic scaling) and **normalize the result**. This is the geometric normal **`Ng`**, and initially **`N = Ng`**. For a polygon mesh, **`Ng`** is the polygon's geometric face normal; if the mesh supplies vertex normals, interpolate them across the grid, transform them with the inverse transpose, normalize them, and use the result as **`N`**. Otherwise, use **`N = Ng`**. The displacement shader therefore always receives an appropriate **undisplaced** normal.

After the displacement shader has run on every grid vertex, recompute the shading normal from the displaced surface. Compute **`dPdu`** and **`dPdv`** by finite differences on the displaced camera-space grid, using neighboring grid vertices along each parameter direction. Use forward differences to follow the reference convention. For `nu` subdivisions along the grid's first direction, `dPdu = nu · (P[i+1][j] - P[i][j])`; on the last column use `nu · (P[i][j] - P[i-1][j])`. Compute `dPdv` the same way along the second direction, using `nv` and a backward difference on the last row. Use the duplicated endpoint vertices at a full periodic seam as ordinary grid boundaries. Keep this grid indexing independent of the shader **`u,v`** fields, which may instead contain interpolated mesh texture coordinates. Then set **`N`** = `normalize(dPdu × dPdv)`. At locations where this cross product is degenerate—such as a sphere's poles, a cone's apex, or the collapsed row of a triangle—fall back to **`Ng`**. Determine which grid locations are degenerate from the **undisplaced** grid, since displacement and floating-point rounding can turn an exactly collapsed row into a tiny, arbitrary tangent and therefore an arbitrary normal. If no displacement shader is bound, you do not need to recompute the normal: for quadrics use **N** = **Ng**, and for a polygon mesh use its interpolated vertex normal when one is supplied. Finally, flip **`N`** toward the eye before running the surface shader; for a mesh with vertex normals, use **`Ng`** to decide which side is facing the eye.

Polygon meshes:
`RiPointsPolygons(npolys, nverts, verts, P, N, st)` draws a mesh of planar, convex polygons. Polygon `i` has `nverts[i]` vertices, and `verts` holds the concatenated vertex indices of all the polygons, so the total mesh has Σ `nverts[i]` entries. **`P`**, **`N`**, and **`st`** are per-vertex arrays indexed by those indices (**`st`** has two floats per vertex and follows `RiTexture()`'s convention: `t = 0` is the top row of the image). **`N`** and **`st`** may be `NULL`. The meshes I give you are already "unwelded," meaning every index refers to one position, one normal, and one texture coordinate, so you never have to deal with per-face-vertex data. Vertices are in object space and go through the current transformation like everything else, and `N` goes through the inverse transpose.

Each polygon is its own primitive: bound it, dice it, shade it, bust it, and free it before moving on to the next one. Its bound is the box around its vertices. Do not try to dice a whole mesh as one grid, and do not keep all of the micropolygons of a mesh around at once (a detailed car can generate tens of millions of micropolygons, and the `McQueen` scene draws it twice).

Dice a polygon as a **bilinear patch**. A quad with vertices **P_0**, **P_1**, **P_2**, **P_3** in cycle order is $P(u,v) = (1-u)(1-v)\,P_0 + u(1-v)\,P_1 + uv\,P_2 + (1-u)v\,P_3$. A triangle is the same patch with one edge collapsed: $P(u,v) = (1-v)[(1-u)\,P_0 + u\,P_1] + v\,P_2$, so the whole `v = 1` row of the grid sits on P_2. This classic trick allows your quadric grid code to work unchanged: an `(nu + 1) × (nv + 1)` grid of vertices, finite differences for `dPdu` and `dPdv`, and the same dice rate as for the quadrics (see "How finely to dice" above). Polygons with more than four vertices are fanned into triangles from vertex 0. The triangle weights are **barycentric**; the quad uses bilinear weights. Interpolate every per-vertex quantity `(N, st)` with exactly the same weights you use for **`P`**. The geometric normal `Ng` of a planar polygon is its face normal, the same at every grid vertex: compute it once and normalize it, using Newell's formula or, for a nondegenerate triangle or quad, $(P_1 - P_0) \times (P_2 - P_0)$ for a triangle or $(P_1 - P_0) \times (P_3 - P_0)$ for a quad, then transform it with the inverse transpose and normalize it again.

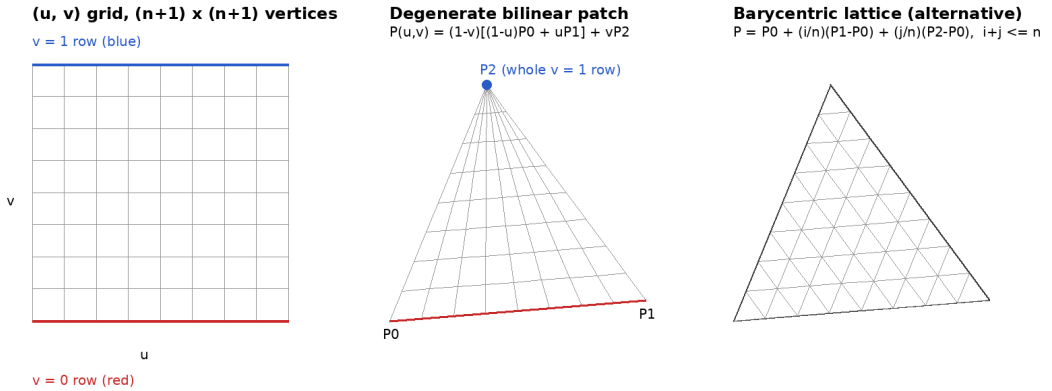


Figure 2. Left: The `(u,v)` grid every primitive is diced into has `(n + 1) × (n + 1)` vertices. **Middle:** A triangle is diced as a bilinear patch whose `v = 1` row collapses onto `P2`. This is what you will implement, and your quadric grid code can dice it unchanged. Its micropolygons get thinner toward `P2`, and the collapsed row is where `dPdu × dPdv` degenerates. **Right:** For comparison only, a barycentric lattice would give evenly sized micropolygons, but it needs a triangular grid instead of the rectangular one you already have, so we don't use it.

Two things to pay attention to:

1. On the collapsed row **`dPdu`** is zero, so `dPdu × dPdv` is degenerate there, exactly like the pole of a sphere, and you fall back to **`Ng`**.
2. Furthermore, the micropolygons in that row have two coincident vertices, so when the hider splits one into two triangles, one of them has zero area. Skip zero-area triangles instead of dividing by their area of zero.

Shading variables on polygons: `P` and `Po` work the same way as for quadrics. If `N` was supplied, the shading normal at each grid vertex is the interpolated vertex normal (inverse transpose, then normalize), which will help the tessellated surface look smooth.

If `N` is `NULL`, then set it to the geometric normal: `N = Ng`. Decide whether to flip `N` toward the eye with the geometric normal, not with `N`: turn `Ng` to agree with `N` (flip `Ng` if `Ng` · `N` < 0), then flip `N` if `Ng` faces away from the eye. An interpolated normal can point slightly away from the eye near the silhouette of a polygon that faces it, and flipping it there lights the silhouette from the wrong side. (RenderMan's `faceforward(N, I)` works the same way.)

If `st` was supplied, the shader's `(u, v)` are the interpolated `st`, so the same texture shader that works on a sphere works on the car. Note that they can fall outside [0, 1] (e.g., the car's tires run from −1 to 2). If `st` is `NULL`, `(u, v)` are the patch parameters, which is handy for a checkerboard on a quad.

The displacement shader sees the interpolated vertex normal, not flipped, so polygons that share a vertex move together along their shared edge. Without vertex normals it sees the face normal, and a displaced mesh opens cracks along its creases. After it runs, recompute `N = normalize(dPdu × dPdv)` exactly as for quadrics. On a mesh this makes the normal faceted again across polygon boundaries when the displacement is large. That is expected.

Other things you will have to implement to get this to work:
`RiColor()` – To specify the current color of the object

You don't have to worry about: splitting, motion blur, depth of field, blobs, concave polygons or polygons with holes (`RiGeneralPolygon`, `RiPointsGeneralPolygons`), per-face-vertex ("`facevarying`") data, sub-division surfaces, NURBS, procedural geometry, level of detail, etc.

4. Hider and image formation
You will need to implement a z-buffer hider similar to that described in the REYES paper by Cook et al. Here, the number of samples `m × n` per pixel should be set by the `RiPixelSamples()` command, and the jitter offsets should be drawn **independently for every pixel**. Basically, you will divide each pixel into `m × n` equal rectangular cells and place one uniformly random sample inside each cell, choosing the offsets independently for every cell of every pixel. When doing this, generate the sample positions once for the current image and reuse them for all primitives, so visibility compares hits at the same positions for all objects.

You can implement this with a `Sample` struct that stores the `x,y` position of the sample and a list of hits for transparency (see Sec. 7). When positioning the random samples, a **fixed random seed** is encouraged for repeatable debugging; you need not match the reference renderer's random samples. Repeating one jitter pattern in every pixel makes the sampling periodic and can reintroduce coherent aliasing.

To determine which samples a micropolygon covers, split each micropolygon into two triangles. For each triangle, compute a raster-space bounding box and intersect it with the image before visiting samples. Test the fixed sample positions within that region for coverage and compute the barycentric weights of each covered sample. Skip zero-area triangles and give shared edges a consistent owner so that a sample contributes at most once for a given surface; this matters especially for transparency. Shade at the grid vertices and interpolate the resulting `Ci` and `Oi` values to each covered sample using the barycentric weights, as described below. As discussed in class, the original 1987 REYES paper instead flat-shades each micropolygon, which you may do as an intermediate debugging step, but **interpolated shading** is required for the final implementation because this is what PRMan does.

Interpolation convention:
You shade at grid vertices, and the hider interpolates their colors, opacities and camera-space depths at each covered sample. Let `b0`, `b1` and `b2` be the triangle's barycentric weights at the sample in raster space. **For this assignment, use linear interpolation in raster space for both projection types:** `z = z0 + b1 · (z1 − z0) + b2 · (z2 − z0)`, and the same form for `Ci` and `Oi`. This is the reference convention and agrees with Sec. 7; perspective-correct `1/z` weighting is not required. Micropolygons are intended to be small, so this approximation is usually adequate, although the dice-rate clamp can produce larger cells. Keep camera-space depth separate from any projected depth value: the hider stores and compares camera-space `z`.

Final image reconstruction:

Continue processing primitives and accumulating visibility information at the fixed sample locations until `RiWorldEnd()` is called. At that point, all geometry in the scene has been rendered, so you can now resolve each sample into a single color and opacity: a sample with no surface hits has `C = (0,0,0)` and `O = (0,0,0)` (note that opacity, unlike standard alpha, has **3 per-color channels**), while a sample with one or more hits is composited from front to back using the procedure described in Sec. 7.

Once every sample has been resolved, apply the required **box filter** to reconstruct the pixels. For each pixel, simply average the resolved colors and opacities of its `m × n` samples, where `m × n` is the sampling rate specified by `RiPixelSamples()`. Thus, for example, with 4 × 4 sampling, each output pixel is the average of its 16 sample colors and 16 sample opacities. Keep the RGB values in their premultiplied form while performing this averaging.

Output:
After reconstruction, each pixel contains a floating-point premultiplied RGB color `C = (Cr,Cg,Cb)` and a three-channel opacity `O = (Or,Og,Ob)`. Our RGBA output buffer has only one alpha value, so compute the output alpha by averaging the three opacities: `A = (Or + Og + Ob) / 3`. Note that all scenes provided use equal opacity in the 3 channels (i.e., `Or = Og = Ob = A`), so the 3-channel opacity only makes a difference if you choose to use different values for the three channels in your own scenes.

Note that all through transparency compositing and box filtering, keep RGB **premultiplied** by opacity. For example, a pure red surface `Ci = (1,0,0)` with 50% opacity in all three channels (e.g., `O = (0.5,0.5,0.5)`) would contribute `C = (0.5,0,0)` and `O = (0.5,0.5,0.5)`. After averaging the opacities, this would give `A = 0.5`. The supplied writer performs the file-format conversion described below.

From `RiWorldEnd()`, hand the supplied `writeImage()` (`write_image()` in Rust) a top-row-first buffer of `xres × yres × 4` floats. For each pixel, store the premultiplied color `C` followed by scalar alpha `A`. **Do not divide by alpha, clamp, or convert to 8 bits yourself.** The writer performs those operations as appropriate for the **output** format and writes the file under `renders/`.

For PNG and TGA: the writer converts to straight RGB by dividing `C` by `A` when `A > 0`, and uses black when `A = 0`. Thus, the red example above is stored as RGB `(1,0,0)` with alpha `0.5`; displaying it over black multiplies by alpha and gives `(0.5,0,0)`.

For JPEG and BMP: the writer stores the premultiplied color `C` without alpha, which is the picture over a black background. After any format-specific alpha conversion, the writer clamps channels to `[0,1]` and rounds them to 8 bits. Filtering and compositing must therefore remain in floating point until the writer is called.

Depth output:
When `RiDisplay(name, "zfile", "z")` is used (or either the type is `"zfile"` or the mode is `"z"`), `RiWorldEnd()` should write a **depth image** instead of a color image. This depth image is used by the supplied code to compute shadow maps.
For each sample, find the nearest surface hit whose opacity is at least 0.5, where the opacity of a hit is the mean of its three interpolated **`Oi`** channels. Ignore hits below this threshold. The depth value for a pixel is then the **smallest camera-space `z`** among the qualifying hits at all of that pixel's samples. If none of the samples has a qualifying hit, store `+infinity`.[1] Thus, mostly transparent surfaces such as glass or the transparent portions of a cutout do not appear in the depth image and therefore do not cast shadows.

Write the depth image using the same output-path convention as a regular image, but in the following raw binary, little-endian format:

```
file signature: 4 bytes = 'R' 'z' '0' '1'
width:          32-bit integer
height:         32-bit integer
depth:          width × height 32-bit IEEE floats, top row first
```

The starter kit's `writeDepth()` in `ri.cpp` (`write_depth()` in Rust's `ri.rs`) writes this file from a buffer of floats, so you only need to fill the depth buffer and call the provided function from `RiWorldEnd()`.

The supplied scene code uses these depth files for shadow mapping [Williams 1978]. A scene first renders a depth image from the light's point of view and then renders the final picture. The shaders read the depth image through the provided `shadowmap.h`, along with the world-to-camera transformation and projection used for the depth pass, so the depth file itself does not need to store any camera information. `shadowmap.h` transforms each shaded point into the shadow-map image using the projection rules from Sec. 2, compares its camera-space depth against the stored value with a small bias, and averages 16 comparisons over a 3 × 3 texel neighborhood ("percentage-closer filtering" [Reeves et al. 1987]) to soften the shadow edges. It then attenuates that light's contribution to `STANDARD` when the point is occluded. The scene `ShadowTest` demonstrates the result; read `shadowmap.h` if you want to see the details. `RadiatorSprings` renders its sun's depth image using an orthographic camera, so your depth output must work with both perspective and orthographic projections.

You don't have to worry about:
- Exposure, tone mapping or gamma correction: hand linear values to the supplied writer, which converts them to 8 bits as described here. Real pipelines apply a display transform (sRGB) first, we skip it here so that your images match the reference renders.
- Other reconstruction filters: the box filter is sufficient.
- Other numbers of color components: assume RGB throughout
- `RiHider()`: RenderMan lets a scene pick the hiding algorithm with it. Ours is always the jittered z-buffer, and no scene calls it, so skip it.
- Shadow lookups: `shadowmap.h` implements them for you. Your renderer only needs to produce the depth file.

[1] Use the IEEE value `INFINITY` from `<math.h>`, `std::numeric_limits<float>::infinity()` in C++, or `f32::INFINITY` in Rust. These are ordinary 4-byte floating-point values that are written like any other, and `shadowmap.h` reads +infinity back as "nothing here."

5. Shading System

RenderMan normally compiles shaders into programs that are executed over many grid vertices in parallel. We will simplify this considerably: in our renderer, a shader is just an ordinary C, C++, or Rust function, depending on your programming language. The renderer calls that function once for each vertex of the diced grid. Specifically, we support two kinds of shaders:

- A **displacement shader**, bound with `RiDisplacement()`, modifies the position **`P`** of a grid vertex to change the shape of the surface.
- A **surface shader**, bound with `RiSurface()`, computes the final color **`Ci`** and opacity **`Oi`** at each grid vertex.

The currently bound shaders are part of the graphics state. When a primitive is rendered, the bound shaders are applied to its micropolygon grid.

Shader context:

In your implementation, the renderer will communicate with a shader through an `RtShaderContext`. Before calling a shader at a grid vertex, your renderer should fill in the appropriate input fields for that vertex. The shader reads those values and writes its results back into the same struct. This context is defined in `Ri.h` (with the same layout in Rust) as:

```
typedef struct RtShaderContext {
    /* inputs, filled by the renderer at each grid vertex */
    RtPoint P;            /* position of grid vertex in camera/eye space */
    RtPoint Po;           /* same point in object space */
    RtNormal N;           /* unit normal, recomputed after displacement */
    RtFloat u, v;         /* surface parameters in [0,1], or a mesh's st */
    RtColor Cs, Os;       /* current RiColor and RiOpacity */
    const void *params;   /* this shader's parameter block, or NULL */
    /* outputs */
    RtColor Ci, Oi;       /* written by surface shaders */
    /* displacement shaders modify P in place */
} RtShaderContext;

typedef void (*RtShaderFn)(RtShaderContext *ctx);

RtVoid RiSurface(RtShaderFn fn, const void *params, size_t nbytes);
RtVoid RiDisplacement(RtShaderFn fn, const void *params, size_t nbytes);
```

Here, **`P`** is the camera-space position of the grid vertex and may be modified by a displacement shader. In contrast, **`Po`** always contains the original, undisplaced object-space position (before the modelview transform). This is useful for procedural patterns such as noise because the pattern remains attached to the object as it moves. **`N`** is the surface normal, also in camera space. A surface shader writes its resulting color and opacity into **`Ci`** and **`Oi`**.

The value of **`N`** depends on which shading pass is running. A displacement shader receives the normal of the original, undisplaced surface: **`Ng`** for a quadric or a polygon without supplied vertex normals, or the interpolated vertex normal for a mesh that supplies them. After displacement, the renderer recomputes **`N`** from the displaced surface as described in Sec. 3, and this new normal is used by the surface shader. Flip **`N`** toward the eye only for the surface-shading pass, not before displacement.

Shader parameters and binding:
Some shaders need parameters chosen by the scene creator, like the size of a checkerboard or the amplitude of a displacement pattern. Each shader may therefore define its own parameter struct. The scene passes a pointer to that struct, along with its size, when the shader is bound.

For example, here are two simple shaders: the first is a displacement shader whose `LiftParams` parameter block specifies how far to move the grid vertex in **`y`**; the second is a surface shader that simply colors the surface green:

```
typedef struct { RtFloat amount; } LiftParams;

/* displacement shader */
void SimpleDisplacementShader(RtShaderContext *c) {
    const LiftParams *p = (const LiftParams *)c->params;
    c->P[1] += p->amount;        /* move the point up in y */
}

/* surface shader */
void SimpleGreenShader(RtShaderContext *c) {
    c->Ci[0] = 0; c->Ci[1] = 1; c->Ci[2] = 0;      /* green */
    /* pass thru opacity */
    c->Oi[0] = c->Os[0]; c->Oi[1] = c->Os[1]; c->Oi[2] = c->Os[2];
}
```

A scene could then bind them as:

```
LiftParams lp = { 2.0 };
RiDisplacement(SimpleDisplacementShader, &lp, sizeof lp);
RiSurface(SimpleGreenShader, NULL, 0);
```

`RiSurface()` and `RiDisplacement()` store the supplied function pointer and copy `nbytes` of the parameter block when the shader is bound. Thus, the parameter struct itself may go out of scope after the binding call. The copy is **shallow**: if the parameter struct contains pointers, the objects those pointers refer to are not copied and must remain valid for as long as the shader may use them.

The shaders bound when a primitive is called are the ones that run on that primitive's grid. A shader that takes no parameters is bound with a `NULL` parameter pointer and zero size (e.g., `RiSurface(SimpleGreenShader, NULL, 0)`). Separately, `RiSurface(NULL, NULL, 0)` has the special meaning of binding the default surface shader `STANDARD`, described below, while `RiDisplacement(NULL, NULL, 0)` removes the currently bound displacement shader.

Shader callbacks are ordinary function pointers. Do not identify shaders with a `switch` statement over the four required shader names: scenes may bind other shader functions as well. Before each shader call, fill the context inputs appropriate to that grid vertex, including **`P`**, **`Po`**, **`N`**, **`u`**, **`v`**, **`Cs`**, **`Os`**, and **`params`**.

Executing shaders:
Although a real RenderMan implementation evaluates many grid vertices in parallel, with those vertices progressing through a shader program in lockstep so that operations such as derivatives can be computed across the grid, **you do not need to implement that execution model**. Within each pass over the grid, do the following for each micropolygon grid vertex sequentially:

1. Fill the `RtShaderContext` for the grid vertex
2. Call the shader function and wait till it fully completes
3. Store its results
4. Move on to the next grid vertex

However, note that you will do **two separate passes over the complete grid**. Do not run the displacement and surface shaders back-to-back at one vertex before moving to the next, because the displaced normal at a vertex depends on the displaced positions of its neighboring grid vertices which would also need to be computed.

Rather, first run the displacement shader on **every** grid vertex, storing its modified **`P`**. Only after the entire grid has been displaced can you recompute **`dPdu`**, **`dPdv`**, and **`N`**, because those quantities depend on neighboring displaced vertices. Then run the surface shader on every grid vertex and store its resulting **`Ci`** and **`Oi`**. The hider later interpolates **`Ci`** and **`Oi`** across each micropolygon.

In other words, the order is:

Build complete grid
↓
Run displacement shader at every vertex
↓
Recompute normals from the displaced grid
↓
Orient N toward the eye for surface shading as described in Sec. 3
↓
Run surface shader at every vertex
↓
Send shaded micropolygons to the hider

For a primitive with no displacement shader, skip both the displacement pass and the normal-recomputation step. Retain the analytic or interpolated normal described in Sec. 3, then orient it for surface shading.

Rust implementation:
A `(NULL, 0)` parameter block should be handled before constructing a Rust slice. If you store copied parameter blocks as owned byte storage, make sure the storage remains suitably aligned for the parameter structs that shaders cast it to. Also, do not hold the renderer's main mutex or an exclusive borrow of its state while invoking an arbitrary shader callback, because the shader may call back into the renderer through functions such as `RiTexture()`.

Shaders to implement:
You will implement **four shaders**, declared in `shaders.h` with exactly these names so that the provided scenes compile against your code:

- `CONSTANT` – The simplest possible shader that copies the values from **`Cs`** to **`Ci`** (and from **`Os`** to **`Oi`**)
- `STANDARD` – The standard lighting model from class (ambient plus Lambertian diffuse plus Phong specular), using **`Cs`** as the surface color and **`Os`** as the opacity. This is also what `RiSurface(NULL, NULL, 0)` binds. The light sources are hardcoded in your shaders file: `ka = 0.15`, `kd = 0.8`, `ks = 0.3`, specular exponent 32, specular color white. Three point lights with no distance falloff, fixed in camera space: a key light at `(-6, 8, -4)` with color (1.0, 0.95, 0.85) and intensity 1.0, a fill light at `(7, 3, -2)` with color (0.6, 0.7, 1.0) and intensity 0.4, and a rim light at `(0, 5, 18)`, white, intensity 0.5. So `Ci = Cs · (ka + kd · Σ max(N · L, 0) · I · C) + ks · Σ max(R ·`

V, 0)^{32} · I · C, summed over the three lights with intensity I and color C, and Oi = Os. Use L = normalize(light_position − P), V = normalize(−P), and the Phong reflection vector R = 2(N·L)N − L. When doing these calculations, make sure you normalize N, L, and V. These are the camera-space conventions used by the supplied scene shaders.

- CHECKERBOARD – A surface shader with parameters typedef struct { RtInt sizex, sizey; RtShaderFn light; const void *light_params; RtShaderFn dark; const void *dark_params; } CheckerParams. It cuts the surface into sizex "checks" across **u** and sizey checks across **v**. The point (**u, v**) is in check (**i, j**) = (⌊u·sizex⌋, ⌊v·sizey⌋), with **u = 1** and **v = 1** counted in the last check. Checks with **i + j** even, including the one at **u = v = 0**, run the light shader, and the others run the dark shader.

  CHECKERBOARD essentially acts as a **dispatcher**: it calls the selected shader on the same RtShaderContext, temporarily setting c->params to that shader's own parameter block (either light_params or dark_params). Thus either branch can be any surface shader, including another CHECKERBOARD. We define a NULL light shader to mean STANDARD, while a NULL dark shader means STANDARD with **Cs** temporarily set to black. For example, CheckerParams cp = { 8, 4 }; gives an 8 × 4 checkerboard of lit checks of the current color and black. If CHECKERBOARD temporarily changes context inputs such as params or **Cs** while calling a subshader, be sure to restore those inputs before returning; the selected shader's output values **Ci** and **Oi** should remain.

- BUMPY – Displacement shader that gives objects a controllably "bumpy" appearance, with parameters typedef struct { RtFloat amplitude; RtInt min_freq_exp, max_freq_exp; } BumpParams. The shader does this by displacing **P** along **N** by h = amplitude · Σ w_e · perlinNoise(2^e · Po) / Σ w_e, summing over e = min_freq_exp … max_freq_exp, with w_e = $2^{(\text{min_freq_exp} - e)}$. Each octave counts half as much as the one below it, and dividing by the total weight keeps |h| ≤ amplitude whatever the octaves, so amplitude is the height of the bumps. perlinNoise() is Ken Perlin's improved noise model [2002] (available here: https://mrl.cs.nyu.edu/~perlin/noise/), with his permutation table that you may copy with attribution (but write the algorithm yourself): signed, about [−1, 1], zero at integer points. Implement this algorithm so that your bumps match the reference images. Evaluate it at **Po**, so the bumps stick to the object. **P** and **N** are in camera space, so amplitude is in camera-space units: scaling geometry or the camera transform does not automatically scale the added displacement amplitude. Scale amplitude explicitly if that is intended.

Keep shader colors in floating point and do not clamp **Ci** at the vertices. Interpolation, opacity compositing and sample averaging must use the original values, including highlights above one. For example, the supplied Material shader divides its specular contribution by opacity so that later compositing preserves the highlight. Only the final output is clamped, by writeImage() (write_image() in Rust).

You don't have to worry about:

- No light shaders! Instead, the light sources are hardcoded directly in the shaders file. Use the light positions and coefficients listed under STANDARD above so that your images match the reference renders.
- No volume shaders or image shaders.
- No shader compilation! Your compiler takes care of that!
- No shader virtual machine or SIMD/lockstep execution. The sequential per-vertex execution model described under **Executing shaders** is sufficient.

6. Textures

Use the provided `stb_image.h` (the `image` crate in Rust) to read a texture from a PNG, JPEG or BMP file and map it onto the surface of the objects. For quadrics, use the micropolygon grid parameters `(u, v)` as the texture coordinates `(s, t)`, where `s` runs left to right across the image and `t` runs top to bottom. `t = 0` is the top row, the first row `stb_image` returns, as in RenderMan. This is the convention expected by the supplied scenes; interpreting the first row as the bottom flips the texture vertically. To stand a textured globe upright, point its z axis down the screen with `RiRotate(90, 1, 0, 0)`, as `UVTest()` does. A nearest-neighbor or bilinear lookup is fine. The reference is bilinear, with s = 0 and s = 1 on the centers of the first and last columns (column = s·(W − 1), row = t·(H − 1)). Leave coordinates in the inclusive interval [0,1] unchanged, so exactly 1 selects the last row or column. For coordinates outside that interval, subtract the floor to repeat into [0,1); this also handles negative coordinates (the car's tire textures require wrapping).

To keep texture handling simple, we will use our own texture-loading and lookup functions (not like RenderMan):

```
RiMakeTexture("my_texture.png", 0);
```

This will load the `"my_texture.png"` texture into the zeroth slot of the texture data structure. There are 8 slots, 0 to 7. The file may be PNG, JPEG or BMP. `RiMakeTexture()` looks for it as given, then in `textures/`. A texture stays loaded until `RiEnd()` or until another `RiMakeTexture()` reuses its slot. Next, we need a way of invoking the texture from within a shader. Implement the lookup function `RiTexture(RtInt slot, RtFloat s, RtFloat t, RtColor result)`, which fills `result` with the color of texture `slot` at (s, t). A shader then uses it like this:

```
/* this shader does a texture lookup on the texture bound at slot zero */
void SimpleTextureShader(RtShaderContext *c) {
    RiTexture(0, c->u, c->v, c->Ci);
    /* pass thru opacity */
    c->Oi[0] = c->Os[0]; c->Oi[1] = c->Os[1]; c->Oi[2] = c->Os[2];
}
```

For a polygon mesh with `st`, `(u, v)` in the context are the mesh's texture coordinates, so this same shader textures the car. (OBJ files put `t = 0` at the bottom of the image. `obj.h` converts, so `st` from `obj.h` is ready to use.)

You don't have to worry about:

- Filtering textures that are accessed from within a shader with some other coordinates than `u,v`

7. Transparency

You will need to implement transparency so that you can render transparent objects. This involves keeping a **depth-sorted collection of surface hits** at each sample in the framebuffer, where each sample stores (`z`, `Ci`, `Oi`) hits sorted by depth from nearest to farthest (in increasing camera-space `z`). You can use a linked list, a pooled list, or another compact depth-sorted structure. When a fully opaque surface hit is generated (one with `Oi = 1` in all three channels), we insert it into its correct position in the list based on depth and discard everything behind it. That's because this hit blocks all farther hits at the same sample location, but nearer transparent hits remain. Also reject a newly generated hit when it is behind a fully opaque hit already in the list; otherwise later geometry can needlessly regrow the list.

When dealing with micropolygons with different opacities at each vertex, interpolate across a micropolygon so that three corners of exactly 1 give exactly 1 even in floating-point arithmetic: use `v0 + w1·(v1 − v0) + w2·(v2 − v0)`, rather than `w0·v0 + w1·v1 + w2·v2` (or compare with a small tolerance). Otherwise an opacity of 1 typically comes out as 0.9999999999999999, the exact test fails, and the lists are never truncated.

For example, a red surface with color (1,0,0) and opacity 0.5 over an opaque blue surface should composite to RGB (0.5,0,0.5) and opacity 1. A single white surface with opacity 0.5 over transparent black should produce composited RGB (0.5,0.5,0.5) and opacity 0.5. If a sample on a quad's shared triangle diagonal becomes 0.75 opaque, the surface was probably inserted twice.

At `RiWorldEnd()`, after all primitives in the current scene have been processed, collapse the hit list at each sample to a single color and opacity per sample by compositing the surface hits from **front-to-back** order (nearest to farthest). To do this, first initialize the accumulated color and opacity to `C = (0,0,0)` and `O = (0,0,0)`. Since the **`Ci`** and **`Oi`** values computed from the shaders and stored at each hit are separate (i.e., **`Ci`** is not yet premultiplied by **`Oi`**), for each hit we update **`C = C + (1 − O) · Ci · Oi`** and **`O = O + (1 − O) · Oi`**. Perform these operations independently for the R, G, and B opacity channels. After each step, **C** is the premultiplied color of all the surfaces processed so far. Once **O = 1** in all three channels, the sample is fully opaque and you can stop; no farther hit can contribute. Keep this accumulated color premultiplied when box-filtering the samples, as described in Sec. 4.

Other things you will have to implement to get this to work:

- `RiOpacity()`

8. Run your system on all provided scenes

Use the supplied scenes throughout development to test your renderer and compare your results with the corresponding outputs in `reference_renders/`. **All provided scenes and both movie sequences are required**, and I expect visually comparable results rather than pixel-for-pixel agreement: small differences caused by independent jitter patterns, the permitted texture-filtering choice, or your chosen maximum dice rate are fine. Use the reference renders to check geometry, transformations, shading, visibility, transparency, and the other required behavior.

Scene organization:

- **Tier 1 – Test scenes:** The small diagnostic scenes in `scenes/1_TestScenes.cpp` are arranged roughly in workflow order. Each scene's source comment explains what it tests and how to diagnose the result. Start with `FirstPrimitive_sphere`, which needs only translation, sphere geometry, projection and the unlit `CONSTANT` shader. The exact selectors and their main checks are listed below. Scenes that combine features are complete only after all of those features work.

  `FirstPrimitive_sphere`, `FirstPrimitive_cylinder`, `FirstPrimitive_cone`, `FirstPrimitive_torus`: one complete quadric, flat colored by `CONSTANT`.

  `OrthoVsPersp_ortho`, `OrthoVsPersp_persp`: the same spheres and cylinder under both projections; final shading uses `STANDARD`.

  `SimpleAnimation`: frame 75 of the ball over the axes. `SimpleAnimationMovie`: all 100 frames.

  `AliasingTest_1x1`, `AliasingTest_4x4`: a fine checkerboard sphere and an edge-on torus, for comparing sample counts.

  `DiceRateTest`: a large sphere, a tiny sphere and a torus, for checking the dice rate and its clamps.

  `QuadricZoo`: partial quadrics, visible and lit interiors, and transparency.

  `Tunnel`: transformed quadrics with checkerboard shading, for placement and transform save/restore.

`ShaderTest`: textures, displacement, checkerboard dispatch and transparency in one regression scene.

`UVTest`: labeled textures on a sphere and cylinder, for checking the orientation of `s` and `t`.

`Earth`: a textured globe with displacement and transparency.

`DisplacementTest`: smooth and displaced spheres, including displacement under a nonuniform scale.

`BumpVsDisplacement`: supplied custom shaders compare normal-only bumps with geometric displacement; a depth pass supplies shadows on a tiled polygon floor.

`Tornado`: a cone with strong `BUMPY` displacement and transparency.

`TransparencyTest`: overlapping partial, semitransparent quadrics.

`TransparencyOrder`: nested transparent shells submitted out of depth order.

`PolygonTest`: a textured quad versus two triangles, a cube, a sphere with vertex normals, a distant triangle, and a displaced quad.

`ShadowTest`: perspective depth output and the supplied shadow-map lookup on a tiled polygon floor.

- **Tier 2 – Intermediate scenes:** `Rocket` combines quadrics, displacement and transparency. `Explorer` adds textures, nested transforms and ground that reaches behind the camera. `McQueen` draws the 19,416-triangle car twice, with textures and glass above a partially transparent floor. `Buzz` uses an 8,148-triangle model and a transparent helmet. `WallE` shows four poses in one image, using rigid parts placed with `RiConcatTransform()`. `Gumbo` shows two 25,500-triangle elephants with vertex normals, procedural wood on the elephants and board, ivory tusks, and a depth pass for shadows.
- **Tier 3 – Cinematic scenes:** `RadiatorSprings`, `AndysRoom`, and `WallEMovie`. These are the final integration tests for the completed renderer. The final render for `AndysRoom` is shown in Figure 3.



Figure 3. The `AndysRoom` scene you will be able to render with your finished REYES rendering system. Woody and Buzz Lightyear characters © Pixar. AI tools were used to help generate this scene.

Running and debugging:

Run individual scenes while developing so that you can isolate problems quickly. During development, you may temporarily reduce `RiFormat()` and `RiPixelSamples()` in a local debugging copy of a scene, but restore the supplied scene files and their final settings before producing your submission renders. When debugging `McQueen`, keep in mind that the apparent reflection in the polished floor is **not** a reflection feature in your renderer: the scene draws the car a second time, upside-down beneath the floor, and that copy is visible through the partly transparent polished checks. Debug `McQueen` first at 320 × 240 with 1 × 1 samples before returning to its supplied final settings.

The executable accepts either a single scene name or a tier selector:

- `reyes <scene>` – render one scene by name
- `reyes test` – render all Tier 1 scenes, including `SimpleAnimationMovie`
- `reyes intermediate` – render all Tier 2 scenes
- `reyes cinematic` – render all Tier 3 scenes, including `WallEMovie`
- `reyes stills` – render every still image but skip the two movie sequences
- `reyes all` – render everything

For final verification, **`reyes all`** should complete successfully. Each scene writes into the corresponding subdirectory under `renders/` (for example, `renders/1_TestScenes/QuadricZoo.png`). The supplied scene picker (`src/main.cpp` or `rust/src/main.rs`) creates the required output directories. `RiDisplay()` only records the relative output name; the supplied image/depth writer prepends `renders/` exactly once when the file is written at `RiWorldEnd()`.

Creating the movies with `ffmpeg`:

Part of the point of making a REYES renderer is to make some movies with it, so there are two movie scenes required as part of this assignment. Once you have implemented the basic shapes and shading, you should be able to run `SimpleAnimation`, which renders a single representative frame for debugging. Once that frame is correct, `SimpleAnimationMovie` renders all 100 individual PNG frames of the animation sequence.

Once these frames are finished rendering, you can use `ffmpeg` to assemble the sequence into a video. `ffmpeg` is a free, separate command-line program. Install it and open a terminal in the starter-kit root (the folder containing `scenes/`, `renders/`, and `reference_renders/`) and run the command below after the frame sequence has been rendered:

```
ffmpeg -framerate 24 -i renders/1_TestScenes/SimpleAnimation/SimpleAnimation_%02d.png
-c:v libx264 -pix_fmt yuv420p -crf 18 renders/1_TestScenes/SimpleAnimation.mp4
```

This command encodes the frames at 24 fps using H.264 and writes `SimpleAnimation.mp4` next to the frame folder, in `renders/1_TestScenes/`. An MP4 video file with H.264 encoding and `yuv420p` pixel format is widely compatible with browsers and standard video players, so it should play in your OS's default player without plug-ins. Congratulations, you've made your first CG movie with your very own renderer!

The second movie is `WallEMovie`, which renders 72 JPEG frames. After rendering all frames, run `ffmpeg` from the starter-kit root (exact command in `README.md`). Fig. 4 shows a frame of it below.

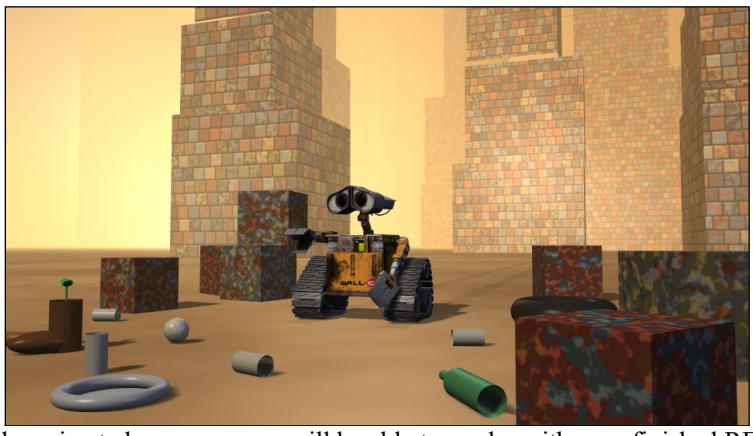


Figure 4. Frame from the animated sequence you will be able to render with your finished REYES rendering system. Wall-E character © Pixar. AI tools were used to help generate this scene.

9. Make a nice final image/movie

Finally, once your renderer is fully working, you will need to create the best possible image/movie using the renderer you've implemented. Remember: aesthetics count! The first step is to configure a nice scene; you are welcome to use the McQueen, Buzz, Wall-E, Woody, or Gumbo models, or use other models you have permission to use and document their sources.[2] If you download other models, you can use AI to convert whatever format they are in to the OBJ format that the supplied loader supports, but verify that your pipeline works correctly before investing time in the scene. You can also use an AI to model the overall scene and "dress" the set.

Once your scene is modeled, you should also write some cool shaders for the objects in the scene (you can use AI for this too, if it is helpful). If you implement extra credit features, please be sure to use them in your final render. Try to iterate as much as possible to produce the best final result. If your scene is animated, you can also optionally produce a movie using the steps described earlier. We will grade your original scene using the 20-point rubric in Sec. 13: 10 points for effective use of your renderer and 10 for composition, materials, lighting and overall visual quality.

10. RenderMan commands that should be implemented

To summarize, in the end your rendering system should implement the following RenderMan commands to work properly:

To set up graphics state

`RiBegin()`
`RiEnd()`
`RiFormat()`
`RiProjection()`
`RiPixelSamples()`
`RiDisplay()` (pictures, and `"zfile"` depth)
`RiFrameBegin()`
`RiFrameEnd()`
`RiWorldBegin()`
`RiWorldEnd()`

Transformations

`RiTransformBegin()`
`RiTransformEnd()`
`RiTranslate()`
`RiRotate()`
`RiScale()`
`RiConcatTransform()`

Primitives

`RiSphere()`
`RiCone()`
`RiCylinder()`
`RiTorus()`
`RiPointsPolygons()`

[2] You can find many great free models at sites like TurboSquid, CGTrader, SketchFab, etc.

Shading

```
RiColor()
RiOpacity()
RiMakeTexture(file, slot)
RiTexture(slot, s, t, result)
RiSurface(fn, params, nbytes)
RiDisplacement(fn, params, nbytes)
perlinNoise(x, y, z)
```
(the scenes call it too)

Use this as your checklist in the end. If I am missing a function here that is needed for the final implementation, let me know!

11. Assumptions

I assumed these in my implementation so you can too! I've been listing assumptions in each of the individual sections, so this is by no means a complete list. It just re-iterates some of the ones I already stated.

- You can assume that there will be no errors in the RenderMan scene descriptions, so you don't have to do any error checking for the user's input (but you should for catching your own mistakes).
- `RiTransformBegin()/RiTransformEnd()` nest, so keep the current transformation on a stack (`Explorer` nests them two deep). They save and restore the transformation only (see Sec. 2).
- You have no depth of field or motion blur, but you can add them for extra credit.
- Your shader code does not have to execute each instruction in parallel mode (as is done by a RenderMan implementation), but you can execute the shader at each grid vertex independently in your chosen language.
- Objects can cross the eye plane. Sec. 3 says what to do (discard micropolygons at $z \le 0.01$, and dice a primitive whose bound reaches $z \le 0.01$ at your maximum rate). You still never split.
- The supported polygon input is planar and convex, and the supplied mesh formats include triangles, quads and n-gons. Do not cull backfaces; use the normal-orientation rules in Sec. 3.
- Polygons can cross the eye plane too, including large backdrop polygons in `Gumbo`. Apply the same micropolygon-discard and maximum-dice rules from Sec. 3 to polygons and quadrics. The `McQueen` floor is tiled to avoid crossing the eye plane, but that is a property of that scene, not an input guarantee.
- Meshes can be large (the car is about 19,000 polygons), so bound and dice per polygon and free each grid before the next.
- I might be missing some other assumptions, so ask if you have any questions.

12. Extra Credit

This can be a tough assignment for some, but there might be some of you that go through it and are looking for a bigger challenge. If you have time, here are a few extra things you can implement for more points. Please indicate in your `README` which extra credit items you attempted and which scenes demonstrate them.

- Implement Bezier bicubic patches as a new primitive. Render a scene using the Utah teapot as an example. You already have bilinear patches, so this is mostly a new dice function. **[7 pts]**
- Implement motion blur by setting a time variable for each sample. Don't forget that you will have to compute the bounding boxes of the objects based on the time as well. **[10 pts]**

- Implement depth of field by giving each sample a lens position and offsetting each micropolygon by its circle of confusion (a function of its depth) scaled by that lens position. **[10 pts]**
- Implement bucketing to accelerate your implementation. **[10 pts]**
- Create a compositing system that allows you to combine RGB+D images rendered by your system (i.e., depth-based compositing [Duff 1985]). Demonstrate your system by rendering some scenes (RGB + depth) and then compositing your renders. **[10 pts]**
- Add depth-based post-process effects, such as fake depth-of-field or fog. **[5 pts]**
- Implement light shaders and demonstrate them working. **[10 pts]**
- Shadows from more than one light, or soft shadows from an area light (several depth maps averaged), building on `shadowmap.h`. Or write your own shadow lookup with a better bias (slope-scaled, or the midpoint of the first two surfaces). **[7 pts]**
- Implement `RiPixelFilter()` with a Gaussian or Catmull-Rom filter wider than one pixel and compare it against the box filter. **[7 pts]**
- Implement splitting. Cut a primitive in half in **u** or **v** (by its parameter ranges) whenever its bound crosses the eye plane, or its dice rate would exceed your maximum, and repeat until every piece can be bounded and diced on its own. Drop the pieces entirely behind the eye. `Explorer` shows what it buys: its ground, a small moon whose bound reaches behind the camera, is otherwise diced at the maximum rate all over, and near the camera its micropolygons are many pixels across (the streaks at the bottom of the reference image). Split, they come out half a pixel. **[10 pts]**
- Adaptive re-dicing: after running the displacement shader, measure the largest micropolygon on screen and, if it exceeds the target size, dice the primitive again at a finer rate. **[5 pts]**
- An environment-map shader: reflect the view direction about **N** and look up a latitude-longitude texture by the reflected direction, which gives mirror-like spheres and chrome at the cost of one texture lookup. **[5 pts]**

13. Grading

Grading is performed by running your code to produce your renders and comparing them with the reference renders, then following up by looking at the source to confirm how each part is done.

Item	Pts	What is graded
Graphics state and camera	8	`RiBegin()`, `RiEnd()`, `RiFormat()`, `RiPixelSamples()`, `RiDisplay()` (format from extension), `RiFrameBegin()`, `RiFrameEnd()`, `RiWorldBegin()`, `RiWorldEnd()`. Both orthographic and perspective projections. `RiTranslate()`, `RiRotate()`, `RiScale()` and `RiConcatTransform()` applied in the right order, and a transformation stack for nested `RiTransformBegin()`, `RiTransformEnd()`.
The four quadrics with partial parameters	10	`RiSphere`, `RiCylinder`, `RiCone` and `RiTorus` as in Fig. 5.3 of the spec, with `zmin`, `zmax`, `thetamax`, `phimin` and `phimax` honored (angles in degrees, and sphere `zmin`/`zmax` clamped to ±radius). Analytic normals through the inverse transpose. The insides of open shapes visible and lit (no backface culling, **N** flipped toward the eye). The four quadrics share these 10 points.
Polygon meshes	7	`RiPointsPolygons`, with each polygon bounded, diced, shaded, busted and freed on its own. Bilinear-patch dicing with a triangle as a collapsed edge, **n**-gons fanned, zero-area triangles skipped. **N** and **st** interpolated with the same weights as **P**, the face normal as **Ng**, and the eye-facing decision made with **Ng**.

Item	Pts	What is graded
Bounding and dicing	8	The eight corners of the bound projected and **n** = 2d from the diagonal, in both **u** and **v**, rounded up and clamped (minimum 2, with 4 allowed for full periodic directions; your chosen maximum), giving an (**n** + 1) × (**n** + 1) grid with **P**, **Po**, **u**, **v** and **Ng** at every vertex, and finite differences for the displaced normal. The eye-plane rule of Sec. 3 (micropolygons with a vertex at **z** ≤ 0.01 discarded, a primitive whose bound reaches it diced at the maximum rate).
Hider and image formation	7	**m** × **n** samples per pixel from `RiPixelSamples`, jittered independently in every pixel. Each micropolygon split into two triangles and tested against the samples its bound touches, with a **z**-buffer per sample. Color and opacity interpolated across each micropolygon from the grid vertices. Box filtering into float, premultiplied RGBA, with alpha the mean of the three opacity channels and zero background opacity. Pass that buffer to the supplied writer for format conversion (Sec. 4).
Depth output	5	`RiDisplay(name, "zfile", "z")` writes the `Rz01` file of Sec. 4: the camera-space **z** of the nearest surface with mean opacity ≥ 0.5, +infinity where nothing is hit, for both perspective and orthographic projections.
Shading system	12	The context struct filled at every grid vertex (**P**, **Po**, **N**, **u**, **v**, **Cs**, **Os**, `params`), with `RiColor` and `RiOpacity` arriving as **Cs** and **Os**. `RiSurface` and `RiDisplacement` copy their parameter block, and `NULL` binds `STANDARD` or unbinds displacement. When displacement is bound, use two passes over the grid: the displacement shader on every vertex (seeing the unflipped **normal: Ng for quadrics and polygons without vertex normals, or the interpolated vertex normal when supplied**), then **N** recomputed from the displaced **dPdu** × **dPdv** with the fallback to **Ng** on degenerate rows, then the surface shader.
The four shaders and `perlinNoise`	8	`CONSTANT` (1). `STANDARD` with exactly the constants and the three lights of Sec. 5, Phong rather than Blinn (2). `CHECKERBOARD` with the parity rule and the two sub-shaders run with their own `params`, so that two different BRDFs work (2). `BUMPY` driven by your own `perlinNoise` [Perlin 2002], normalized octaves, evaluated at **Po**, matching the reference bumps (3).
Textures	5	`RiMakeTexture` into 8 slots (PNG, JPEG or BMP, found as given or in `textures/`) and `RiTexture` with a bilinear or nearest lookup, t = 0 on the top row, repeating outside [0, 1], the same shader working on a quadric's (**u**, **v**) and a mesh's **st**.
Transparency	8	`RiOpacity`. A depth-sorted collection of (**z, Ci, Oi**) hits at every sample, truncated behind a fully opaque hit, with interpolation that preserves an opacity of 1. Hits are composited **front to back**, independently per color channel, using **C ← C + (1 − O) · Ci · Oi** and **O ← O + (1 − O) · Oi**, starting from zero. Both updates use the previous value of **O**. Stop when **O = 1** in all three channels.
Nice final image/video	20	A nice image/video of an original scene, in the scene table and rendered at the resolution in its scene file. Checked on your registered original scene. Half for how much of your renderer it uses (meshes, textures, shaders, displacement, transparency, shadows), half for how it looks: aesthetics count.
Build, README and AI-use statement	2	Builds with the documented platform-specific commands from Sec. 1 (or `cargo build --release` from `rust/`) against the supplied final scene settings and the unchanged public declarations in `Ri.h` and `shaders.h` (or their Rust equivalents). The renderer implementation in `ri.rs` is yours to edit. A `README` with the build and run commands, the extra credit attempted and the scenes that show it, model sources, and the AI-use statement.
Total	**100**	

Points will be deducted for:

- Bad sloppy code
- Unauthorized use of AI
- Missing documentation

14. Tips and advice

- Get started soon – don't wait until the last minute: you won't finish!
- To help you plan and complete the assignment, I've put together a step-by-step guide that basically walks you through the whole assignment (see `HW1_SuggestedWorkflow.pdf`).
- Ask me if you have questions. I'll try to reply as soon as possible. You can also ask an LLM to help explain stuff to you, break down the tasks, and brainstorm approaches (within the permissions and restrictions in Sec. 16).
- Make sure you use the Canvas discussion board to discuss questions/problems with other students. You are welcome to discuss approaches to problems, etc., just don't give away your source code to others. In the end of the course, I might "bump up" the grade slightly for students that have been participating heavily on the board, both asking questions as well as providing help for other students.
- The `McQueen.cpp` scene can be very slow if you are careless. At 960 × 720, the car can generate tens of millions of micropolygons and is drawn twice (car + mirrored version). Runtime depends on hardware, build optimization, dice limits and data structures, but in my implementation it rendered in around 40 seconds on one core. When working with this scene, debug at 320 × 240 with 1 × 1 samples first, keep the per-micropolygon work tight (the bounding box of a half-pixel micropolygon should only ever touch a handful of samples), and only go to full resolution once the picture is right. If you want it fast, bucketing is in the extra credit.

15. Submission Instructions and Deliverables

Write your renderer in **C, C++, or Rust**. For C and C++, your project must build with the provided `CMakeLists.txt`, using the platform-specific configure/`build` commands in Sec. 1. For Rust, it must build with `cargo build --release` from `rust/` using `Cargo.toml` and `build.rs` with your completed `src/ri.rs` and `src/shaders.rs` implementations; the build compiles the C++ scene files and links them against your renderer. Use a supported platform (Linux, macOS or Windows) and document the platform and complete configure/build/run commands you tested. For a Windows Visual Studio build, select the Release configuration with `cmake --build build --config Release`. The provided scene interface must remain portable. The scene files are the interface: your renderer must expose exactly the `Ri*` functions and the shader context struct in `Ri.h` (or `ri.rs`), so that the provided scenes compile and run unchanged. No rendering, rasterization, or geometry libraries of any kind (the provided `obj.h` is the one exception, and it only reads files). Image input and output go through the provided `stb` headers (or, in Rust, the image crate, which the kit's `Cargo.toml` already lists).

Include a `README` with the build command and the run commands for the required scene selectors and your original scene, and include the source, build files, `Cargo.lock` for Rust, and required model/texture assets in the zip; omit generated `build/` and `target/` directories. Your project must build with the documented command or we cannot grade it. Add your own scenes as scene files by creating a new `.cpp` directly inside `scenes/`, and then adding the file to `CMakeLists.txt` (the Rust build discovers files directly in `scenes/` automatically), declaring the scene function in `scenes/0_scene_table.cpp` and adding its name/function/tier entry to `RI_SCENES`, so that your original scene can be rendered by name through the same executable.

Zip up the source code and deliverables and submit them on Canvas before the deadline. You will need to submit the following items by the deadline:

- Source code for your project, including the build files (`CMakeLists.txt` or `Cargo.toml`) and the `README`.
- **Rendered outputs:** Submit all still images and all numbered frames for the provided scenes in Sec. 8, together with the encoded **`SimpleAnimation.mp4`** and **`WallEMovie.mp4`** videos. Also include the final image or movie for your original scene from Sec. 9. Use the supplied final resolutions and sample counts for the provided scenes, and preserve the output directory structure

under **`renders/`**. Include the source for your original scene and all shaders you wrote. Note that grading starts from the images, so a missing image is a missing feature.
- Any extra credit you did (Sec. 12).

Late submissions use your late days first (five late days shared across HW1 to HW4, none for the final project). After those are used up, late work loses 20% per day.

16. AI policy

In this course, you can use AI tools like you would a classmate or TA: they may explain concepts, help you debug, and give you feedback/advice on your approach. They may also write project setup, build files, image I/O, timing and plotting scripts to accelerate your workflow. They may help interpret compiler/runtime errors and suggest debugging strategies, but any changes to the required renderer algorithms must be implemented by you. But they may not write the renderer for you: the primitives (quadrics and polygons), bounding, dicing, shading (the four shaders and the noise function), the hider, filtering, depth output, and transparency are all yours to implement. After all, I need you to learn how to do this stuff yourself!

However, you can also use AI for artistic purposes, such as the creation of scenes, textures, shader parameters, or converting a model you found into OBJ form. I really encourage you to use it in this way, since after all, if you're working as a rendering engineer at a production studio you'll have talented artists helping you with your work. In this case, the AI can play the role of the "talented artist." As described in Sec. 9, AI may also help write additional artistic shaders for your original scene. This permission does not extend to the four required shaders, `perlinNoise()`, or any other required renderer algorithms.

Finally, include a short AI-use statement in your `README` describing what you used AI for. You must be able to explain any line of your code if asked.

Best of luck!